Advanced Analysis of a Process Injector

Wednesday, 31 December 2025 17.15

- In this analysis we are going to do the advanced analysis on the malware called malware.stage0.exe.
- Before doing the advanced analysis, first proceed with the basic static analysis and let's see what we sort of indicators we can catch.

```
               2 Dir(s)  30 475 685 888 bytes free

C:\Users\Immune\Desktop\Malware
λ Floss.exe Malware.stage0.exe.malz > stage0floss.txt
INFO: floss: extracting static strings
finding decoding function features: 100%|
INFO: floss.stackstrings: extracting stackstrings from 284 functions
INFO: floss.results: Error: unhandled exception: [
extracting stackstrings: 100%|
INFO: floss.tightstrings: extracting tightstrings from 7 functions...
extracting tightstrings from function 0x40b320: 100%|
INFO: floss.string_decoder: decoding strings
emulating function 0x40b320 (call 1/1): 100%|
INFO: floss: finished execution after 64.45 seconds
INFO: floss: rendering results

C:\Users\Immune\Desktop\Malware
```

Some interesting indicators.

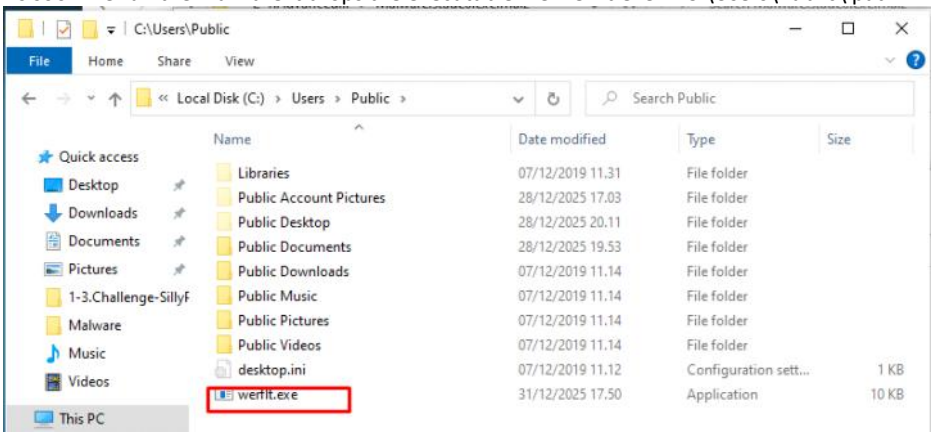
- C:\Users\Administrator\source\repos\CRTInjectorConsole\Release\CRTInjectorConsole.pdb
  -@C:\Users\Public\werflt.exe
  -@C:\Windows\SysWOW64\WerFault.exe
  -@C:\Users\Public\werflt.exe

- Doing the floss.exe to see the overview of string of our malware.
- From the indicators which we have found during the basic static analysis, it tells us that the .exe file called WerFault.exe and werflt.exe are two different interesting file. It's fascinating that their name looks similar but not exactly same, which is very eye catching and abnormal.

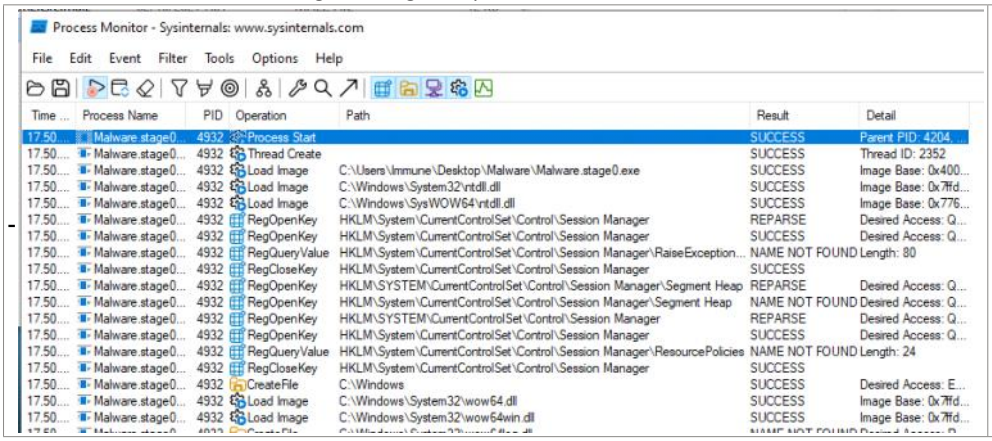
Basic dynamic analysis of malware.

******Note: Run malware in safe environment/sandbox primarily designed for malware analysis which cannot talk to the outside world.*****

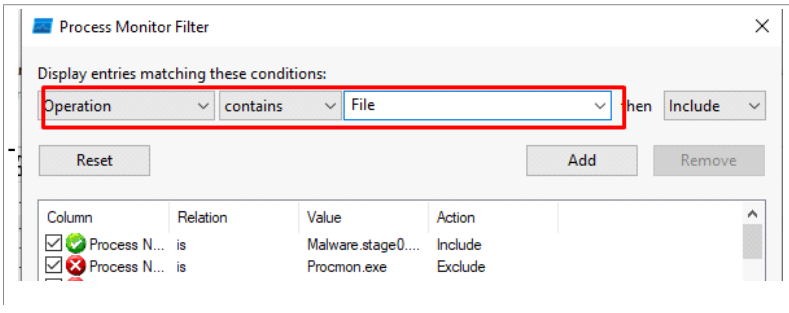
- As soon we run the malware it drops the executable file werflt.exe in C:\Users\Public\ path.



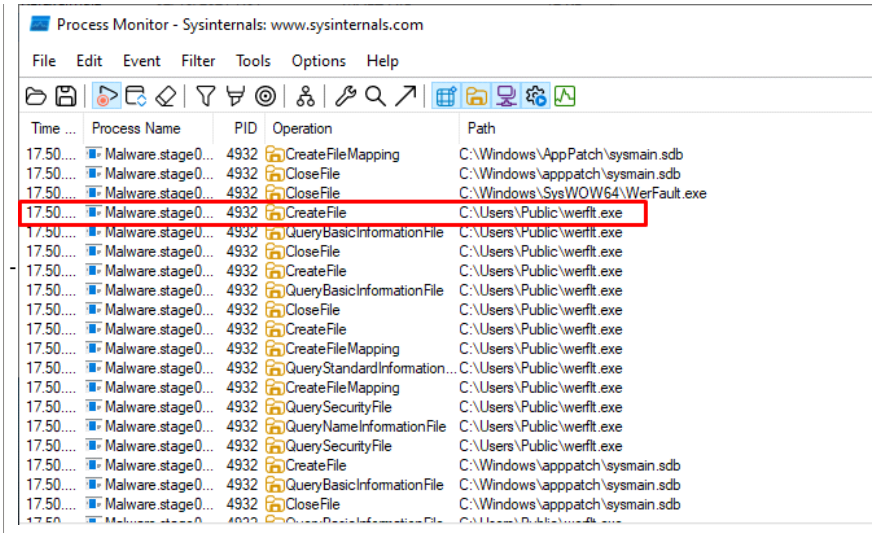
- For host based indicators, let's get through the procmon.



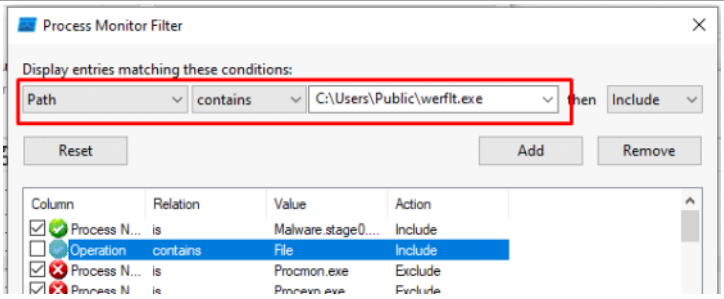
- Filter the process name to find what actually it is doing.

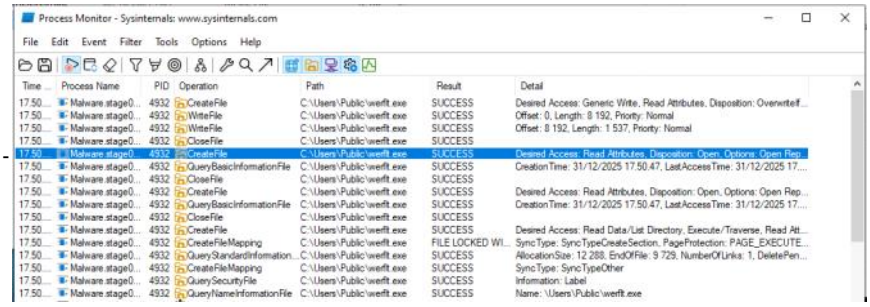


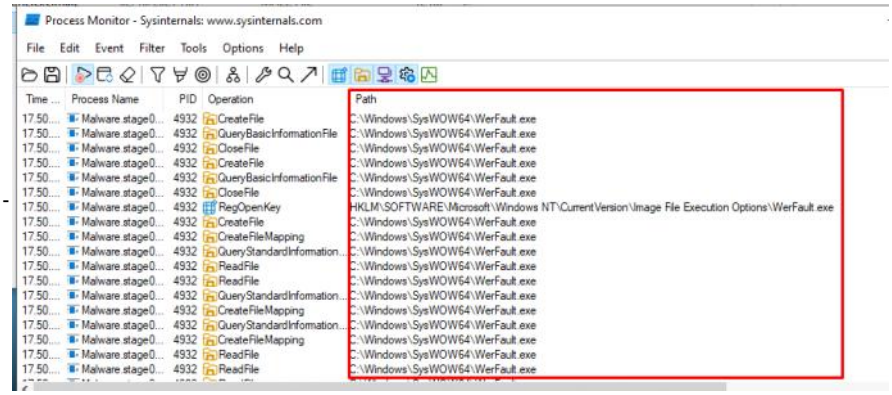
- Filter with operation that contains File
- Werflt.exe file is dropped in the Public folder.

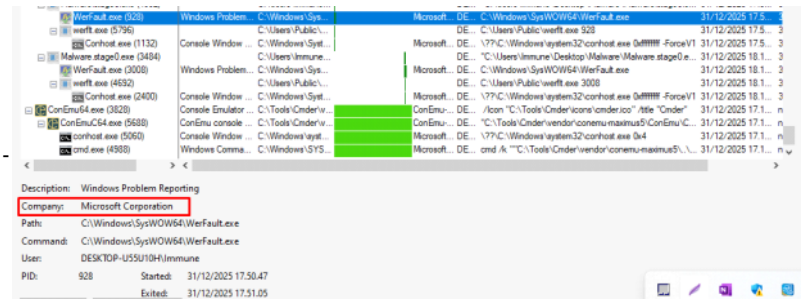


- From path

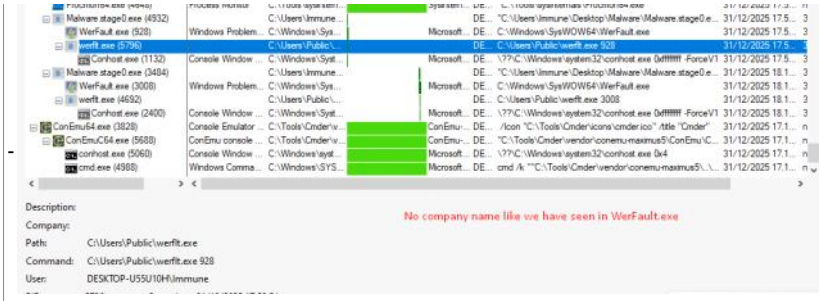




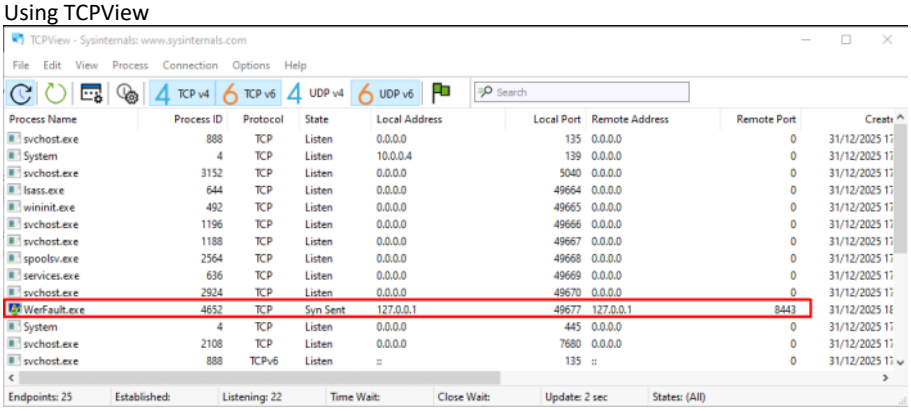




- Filter with the path name.
- Check the authenticity of the exe file.
- Seems like there is nothing about werflt.exe however we have information on Werfault.exe, which also appears in our findings, which appears to be the legitimate exe file and it is the error reporting fault from Microsoft.
- Werflt is an unpacked binary and not the WerFault.exe from microsoft.

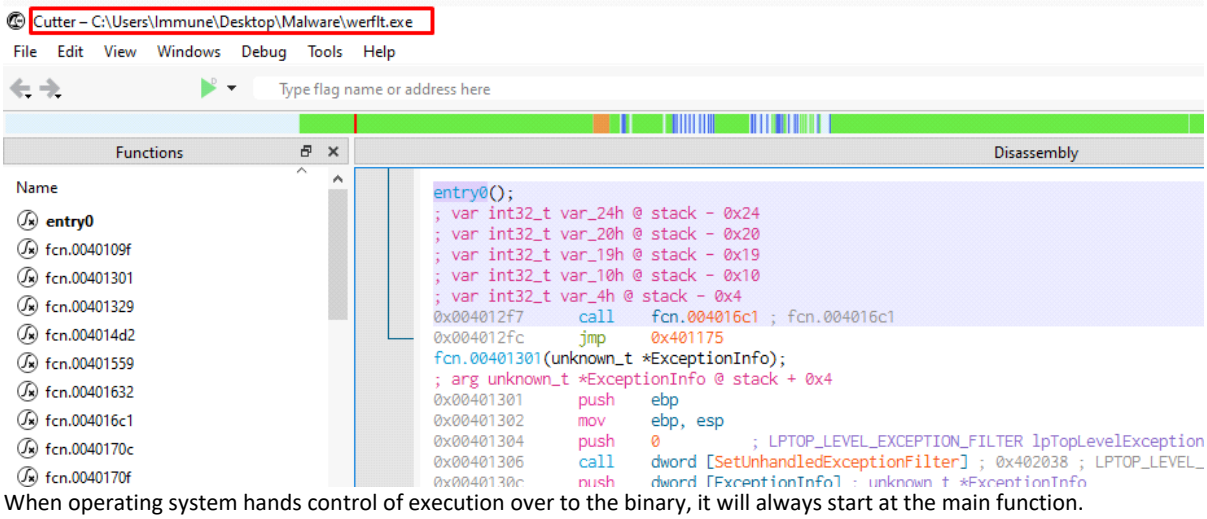
- 

Network Based Indicators

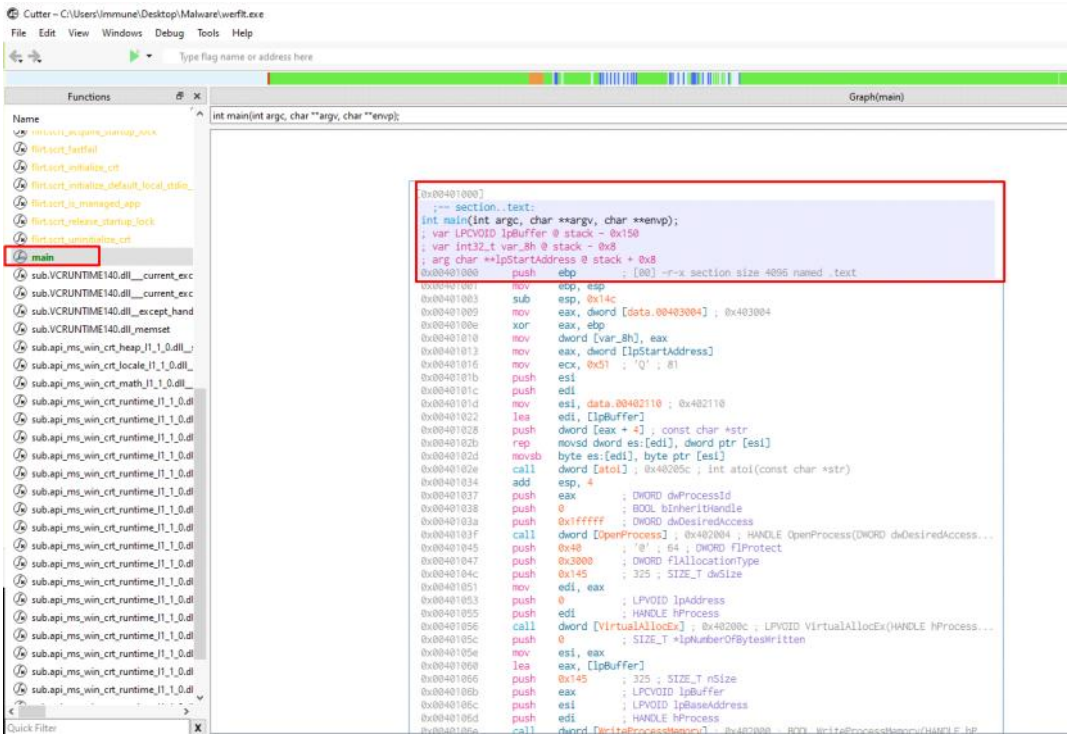
- Using TCPView
- 

- The WerFault.exe spawns an open connection for a brief moment on 8443 on the local host. Its opening or attempting to open a connection on port 8443.

Advanced Static analysis of the werflt.exe

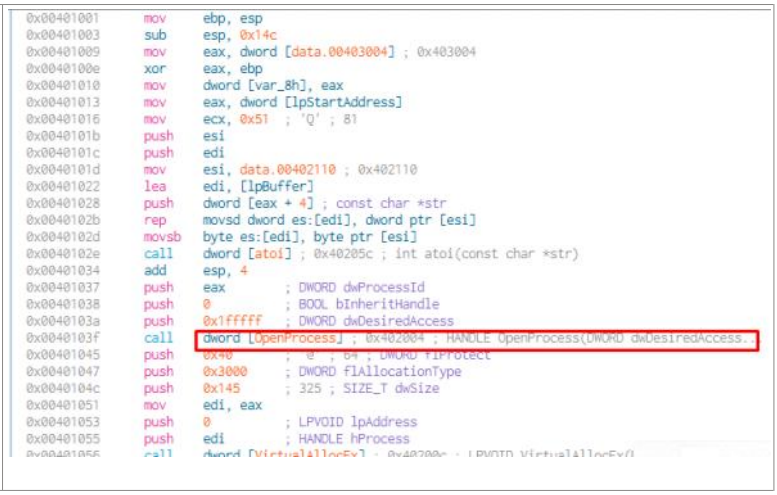
- Tools used cutter.
- 

- When operating system hands control of execution over to the binary, it will always start at the main function.



- When we inspect what we see in the main function during our analysis of main function, we come up with the classic pattern for a create remote thread process injection.
- Process injection is the common TTP for malicious actors, and effectively and they are able to open up another process that's running on the host and inject code right into that process and have that code run inside the process as if it was part of that program in the first place.
- This is a common evasion technique, because the code that runs inside of the process has all of the attributes and metadata and lives inside of that process in the first place.

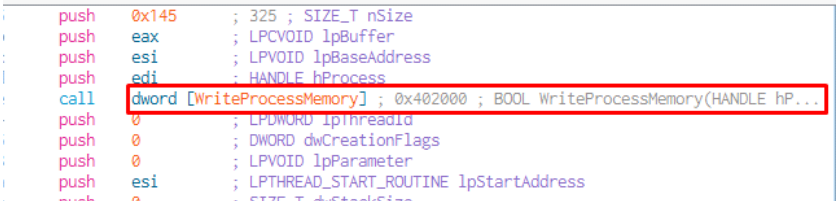
OpenProcess API



- Handle process is end up passing up to the main, argc ends up being the process handler.
- Basically thid program is given a handle process, loads it in eax, it uses that to open up access to the specified process that has given as an argument. Then it takes the same process ID and load it into EDI register.
- Mov edi, eax. Here eax was the handle process. Its holding right in the edr register to reuse again in virtualallocex api.



- Allocate the memory inside that process.
- Another API WriteProcessMemory also uses the handleprocess.



Steps Happened

- Opening up a process.
- Allocating an area of memory inside of this process. Take bytes from lpbuffer variable and writing them into that process in the allocated memory.
- Next API is called Create Remote Thread.

- Next API is called CreateRemote Thread.

```
push    0x145       ; 325 ; SIZE_T nSize
push    eax         ; LPCVOID lpBuffer
push    esi         ; LPVOID lpBaseAddress
push    edi         ; HANDLE hProcess
call    dword [WriteProcessMemory] ; 0x402000 ; BOOL WriteProcessMemory(HANDLE hP...
push    0           ; LPDWORD lpThreadId
push    0           ; DWORD dwCreationFlags
push    0           ; LPVOID lpParameter
push    esi         ; LPTHREAD_START_ROUTINE lpStartAddress
push    0           ; SIZE_T dwStackSize
push    0           ; LPSECURITY_ATTRIBUTES lpThreadAttributes
push    edi         ; HANDLE hProcess
```

- It uses the lpbuffer, it is the variable in the very top of the program.

```
[0x00401000]
   ;-- section..text:
int main(int argc, char **argv, char **envp);
; var LPCVOID lpBuffer @ stack - 0x150
; var int32_t var_8h @ stack - 0x8
; arg char **lpStartAddress @ stack + 0x8
0x00401000      push    ebp         ; [00] -r-x section size 4096 named .text
0x00401001      mov     ebp, esp
```

- The grand final API is CreateRemoteThread

```
push    edi         ; HANDLE hProcess
call    dword [CreateRemoteThread] ; 0x402010 ; HANDLE CreateRemoteThread(HANDLE ..
push    edi         ; HANDLE hObject
call    dword [CloseHandle] ; 0x402008 ; BOOL CloseHandle(HANDLE hObject)
mov     ecx, dword [var_8h]
xor     eax, eax
pop     edi
xor     ecx, ebp
pop     esi
call    fcn.0040109f ; fcn.0040109f
mov     esp, ebp
pop     ebp
ret
```

- Creating a remote thread inside of a remote process. And do that by handing it from the arguments of the main function.
- This is a common way for shellcode to be injected into a remote process because you can write your shell code into the body of the program and then copy those bytes into the remote process, write them into memory and then execute them with a new thread via tha CreateRemoteThread API.

Process Hacker

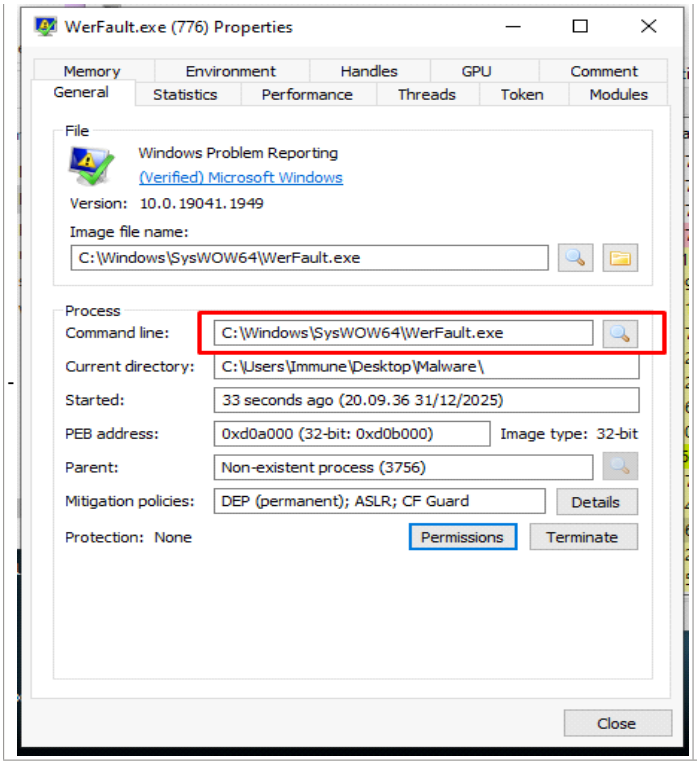
- Let's see it in process hacker. First let's ncat and listen in the port 8443.

```
C:\Users\Immune\Desktop\Malware
λ ncat -nvlp 8443
Ncat: Version 7.98 ( https://nmap.org/ncat )
Ncat: Listening on [::]:8443
Ncat: Listening on 0.0.0.0:8443
Ncat: Connection from 127.0.0.1:49678.
```

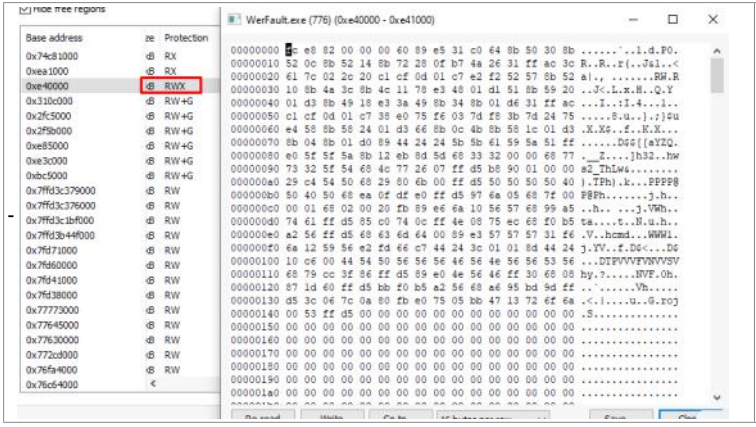
- Run the malwarestage0.
- Then from process hacker.

explorer.exe	4204	1,59		89,72 MB	DESKTOP-U5...\Immune	Windows Explorer
SecurityHealthSystray.exe	5360			1,6 MB	DESKTOP-U5...\Immune	Windows Security notification...
VBoxTray.exe	5460	0,01	56 B/s	3,01 MB	DESKTOP-U5...\Immune	VirtualBox Guest Additions Tra...
msedge.exe	5648	0,16	72 B/s	42,12 MB	DESKTOP-U5...\Immune	Microsoft Edge
Zoomlt64.exe	3280			1,78 MB	DESKTOP-U5...\Immune	Sysinternals Screen Magnifier
ConEmu64.exe	3828	0,60		7,27 MB	DESKTOP-U5...\Immune	Console Emulator (x64)
ConEmuC64.exe	5688	0,18	10,51 kB/s	2,23 MB	DESKTOP-U5...\Immune	ConEmu console extender (x64)
conhost.exe	5060	0,10	3,48 kB/s	2,65 MB	DESKTOP-U5...\Immune	Console Window Host
cmd.exe	4988			19,08 MB	DESKTOP-U5...\Immune	Windows Command Processor
ncat.exe	1160			15,8 MB	DESKTOP-U5...\Immune	ShimGen generated shim - Ch...
ncat.exe	5236			1,79 MB	DESKTOP-U5...\Immune	
ProcessHacker.exe	4240	3,02		12,34 MB	DESKTOP-U5...\Immune	Process Hacker
WerFault.exe	776			1,73 MB	DESKTOP-U5...\Immune	Windows Problem Reporting

- WerFault.exe spawned in a suspended state. While that spawns in a suspended state, the rest of the program is kicked off. The PID of where fault I,e 776, is handed back to unpacked program werflt.exe and when werflt is executed, it takes the process id 776 and opens the process up, bypassing it in processhandle(776) and allocates the area of memory inside that process, it writes the shell code that it is using into that process and creates the remote thread and execute the reverse shell back.



- This is the completely legitimate exe file from Microsoft.



In the memory tab, when the area for memory allocation is done, it is done with the full read, write and executable permission. i.e RWX as seen in the given screenshot. In the right side of the hexdump is the actual byte of shell code.

- That is how a basic create remote thread process injector looks like from an assembly level.